

고급과제 2

LLM 코드 리뷰 및 개선 보고서

학번 : 2026406017

이름 : 이규환

1. LLM 입력 프롬프트

본 과제에서는 아래와 같은 프롬프트를 LLM(Chat GPT)에 입력하여 명령어 기반 도서 대출 관리 시스템을 생성하였다.

프롬프트

명령어 기반 도서 대출 관리 시스템을 만들거야

사용할 명령어는 다음과 같아

add, borrow, return, status, user, list, Exit

기본적으로 무한반복문을 사용해서 Exit 가 입력되기 전까지 반복 실행하도록 할거야

각 명령어의 입력 서식과 동작을 알려줄게

-> add 책이름 수량

:책과 책의 수량을 등록해 만약 같은 제목을 가진 책이 이미 등록되어있다면 수량을 추가해

-> borrow 사용자 책이름

:책이 등록되어있고 수량이 남아있으면 같은 사용자가 같은 책을 대출중이지 않을때 대출 (대출시 책 수량 - 1), 실패시 ERROR + 사유 출력

-> return 사용자 책이름

:실제로 그 사용자가 대출 중인 책만 반납 가능(반납시 책 수량 + 1), 실패시 ERROR + 사유 출력

-> status 책이름

:남은 수량을 출력, 존재하지 않는 책일때 ERROR 출력

-> user 사용자

:사용자의 대출 목록 출력, 대출 목록이 없으면 EMPTY, 사용자가 존재하지않으면 ERROR 출력

-> list

:전체 책 목록을 등록 순서대로 출력

-> Exit

:프로그램 종료

조건이 있어

1. 책과 사용자 이름에는 공백이 없음
2. 수량은 0 이상의 정수
3. 잘못된 명령어 형식을 입력받으면 ERROR 출력
4. eval() 사용 금지
5. 함수를 통한 기능 분리

2. 프롬프트 입력 후 LLM 첫 출력 코드

LLM 이 처음 생성한 코드는 별도의 수정 없이 그대로 저장하여 레퍼런스 코드로 사용하였다.

```
# 고급과제 2 수정 전

# 책 이름과 남은 수량 저장
books = {}

# 책 등록 순서 저장
book_order = []

# 사용자별 대출 목록 저장
users = {}

# 책 등록
def add(cmd):
    # 명령어 형식 확인
    if len(cmd) != 3:
        print("ERROR")
        return

    title = cmd[1]

    # 수량이 정수인지 확인
    try:
        count = int(cmd[2])
        if count < 0:
            print("ERROR")
            return
    except:
        print("ERROR")
        return

    # 이미 등록된 책이면 수량 추가
    if title in books:
```

```

        books[title] += count
    else:
        # 새로운 책 등록
        books[title] = count
        book_order.append(title)

# 책 대출
def borrow(cmd):
    # 명령어 형식 확인
    if len(cmd) != 3:
        print("ERROR")
        return

    user = cmd[1]
    title = cmd[2]

    # 책이 존재하는지 확인
    if title not in books:
        print("ERROR 책이 존재하지 않습니다.")
        return

    # 남은 수량 확인
    if books[title] == 0:
        print("ERROR 수량이 없습니다.")
        return

    # 처음 대출하는 사용자 등록
    if user not in users:
        users[user] = []

    # 같은 책 중복 대출 방지
    if title in users[user]:
        print("ERROR 이미 대출 중입니다.")
        return

    users[user].append(title)
    books[title] -= 1

# 책 반납
def return_book(cmd):
    # 명령어 형식 확인
    if len(cmd) != 3:
        print("ERROR")
        return

    user = cmd[1]
    title = cmd[2]

```

```

# 사용자 존재 여부 확인
if user not in users:
    print("ERROR 사용자가 존재하지 않습니다.")
    return

# 실제 대출한 책인지 확인
if title not in users[user]:
    print("ERROR 대출하지 않은 책입니다.")
    return

users[user].remove(title)
books[title] += 1

# 책의 남은 수량 조회
def status(cmd):
    if len(cmd) != 2:
        print("ERROR")
        return

    title = cmd[1]

    if title not in books:
        print("ERROR")
        return

    print(books[title])

# 사용자 대출 목록 조회
def user(cmd):
    if len(cmd) != 2:
        print("ERROR")
        return

    name = cmd[1]

    if name not in users:
        print("ERROR")
        return

    if len(users[name]) == 0:
        print("EMPTY")
        return

    for book in users[name]:
        print(book)

# 등록된 모든 책 출력
def list_book(cmd):

```

```
if len(cmd) != 1:
    print("ERROR")
    return

if len(book_order) == 0:
    print("EMPTY")
    return

# 등록된 순서대로 출력
for book in book_order:
    print(book, books[book])

# Exit 입력 전까지 반복 실행
while True:
    command = input().split()

    # 빈 입력 처리
    if len(command) == 0:
        print("ERROR")
        continue

    # 프로그램 종료
    if command[0] == "Exit":
        break

    # 명령어에 따라 함수 호출
    elif command[0] == "add":
        add(command)

    elif command[0] == "borrow":
        borrow(command)

    elif command[0] == "return":
        return_book(command)

    elif command[0] == "status":
        status(command)

    elif command[0] == "user":
        user(command)

    elif command[0] == "list":
        list_book(command)

    # 존재하지 않는 명령어 처리
    else:
        print("ERROR")
```

3. 오류 및 개선점 분석

3.1 프로그램 동작 확인

생성된 코드를 직접 실행하여 각 명령어(add, borrow, return, status, user, list)를 테스트하였다.

테스트 결과 대부분의 기능은 요구사항에 맞게 정상적으로 동작하였다. 도서 등록, 대출, 반납, 남은 수량 조회, 사용자별 대출 목록 조회, 전체 도서 목록 출력 등의 기능은 오류 없이 수행되었으며, 잘못된 명령어 형식이나 존재하지 않는 도서에 대한 예외 처리도 구현되어 있었다.

하지만 프로그램을 실제로 사용하면서 몇 가지 개선이 필요한 부분을 확인하였다.

3.2 성공 여부 출력 기능 추가

처음 생성된 코드에서는 명령이 정상적으로 수행되어도 아무런 출력이 없었다.

예를 들어,

```
add Python 3
```

을 입력하면 책은 정상적으로 등록되지만 화면에는 아무런 메시지가 출력되지 않았다.

또한,

```
borrow Kim Python
```

역시 정상적으로 대출이 완료되었는지 사용자가 즉시 확인하기 어려웠다.

사용자의 편의성을 높이기 위해 각 기능이 정상적으로 수행되었을 경우 성공 메시지를 출력하도록 수정하였다.

예를 들어,

```
[ADD SUCCESS] Python 3 권 등록 완료
```

```
[BORROW SUCCESS] Kim 님이 Python 을 대출했습니다.
```

```
[RETURN SUCCESS] Kim 님이 Python 을 반납했습니다.
```

와 같이 수행 결과를 바로 확인할 수 있도록 개선하였다.

3.3 출력 형식 개선

기존 코드에서는 일부 출력이 지나치게 단순하였다.

예를 들어 status 명령을 실행하면

```
2
```

와 같이 숫자만 출력되었다.

사용자가 어떤 값을 의미하는지 쉽게 알 수 있도록 다음과 같이 수정하였다.

```
Python : 2 권 남음
```

```
또한 list 명령 역시
```

기존

```
Python 3
```

```
C 2
```

에서

개선 후

```
===== BOOK LIST =====
```

```
Python : 3 권
```

```
C : 2 권
```

```
=====
```

형태로 변경하여 가독성을 높였다.

3.4 프로그램 시작 화면 추가

기존 프로그램은 실행과 동시에 입력을 받기 때문에 사용할 수 있는 명령어를 사용자가 기억하고 있어야 했다.

이를 개선하기 위해 프로그램 시작 시 안내 문구를 출력하도록 수정하였다.

===== 도서 대출 관리 시스템 =====

사용 가능한 명령어

add
borrow
return
status
user
list
Exit

=====

프로그램의 기능을 처음 사용하는 사용자도 쉽게 사용할 수 있도록 개선하였다.

3.5 코드 구조 개선

처음 생성된 코드는 프로그램이 전역 영역에서 바로 실행되는 구조였다.

이를 main() 함수로 분리하여 프로그램의 실행 흐름을 명확하게 구성하였다.

또한 함수 이름도 보다 의미가 잘 전달되도록 수정하였다.

예를 들어,

수정 전	수정 후
list_book()	show_book_list()
status()	check_status()

와 같이 함수의 역할이 이름만 보아도 이해될 수 있도록 변경하였다.

3.6 코드 가독성 향상

처음 생성된 코드에는 주석이 거의 존재하지 않아 프로그램의 흐름을 이해하기 어려웠다.

이를 개선하기 위해 각 함수의 역할과 주요 기능을 설명하는 간단한 주석을 추가하였다.

예를 들어,

```
# 책 등록
```

```
def add_book():
```

```
# 책 대출
```

```
def borrow_book():
```

```
# 전체 도서 목록 출력
```

```
def show_book_list():
```

과 같이 함수의 목적을 쉽게 이해할 수 있도록 수정하였다.

4. 개선한 코드

```
# 고급과제 2 (수정 후)
```

```
# 책 이름과 남은 수량 저장
```

```
books = {}
```

```
# 책 등록 순서 저장
```

```
book_order = []
```

```
# 사용자별 대출 목록 저장
```

```
users = {}
```

```
# 책 등록
```

```
def add_book(command):
```

```
    # 명령어 형식 확인
```

```
    if len(command) != 3:
```

```
        print("ERROR : 잘못된 명령어 형식입니다.")
```

```
        return
```

```
    title = command[1]
```

```
    # 수량이 정수인지 확인
```

```
    try:
```

```
        count = int(command[2])
```

```
        if count < 0:
```

```
            print("ERROR : 수량은 0 이상의 정수여야 합니다.")
```

```
            return
```

```
    except:
```

```

        print("ERROR : 수량은 정수여야 합니다.")
        return

# 이미 등록된 책이면 수량 추가
if title in books:
    books[title] += count
    print(f"[ADD SUCCESS] {title}의 수량이 {books[title]}권이 되었습니다.")
else:
    # 새로운 책 등록
    books[title] = count
    book_order.append(title)
    print(f"[ADD SUCCESS] {title} {count}권 등록 완료.")

# 책 대출
def borrow_book(command):

    if len(command) != 3:
        print("ERROR : 잘못된 명령어 형식입니다.")
        return

    user = command[1]
    title = command[2]

    # 책 존재 여부 확인
    if title not in books:
        print("ERROR : 존재하지 않는 책입니다.")
        return

    # 남은 수량 확인
    if books[title] == 0:
        print("ERROR : 남은 수량이 없습니다.")
        return

    # 새로운 사용자 등록
    if user not in users:
        users[user] = []

    # 같은 책 중복 대출 방지
    if title in users[user]:
        print("ERROR : 이미 대출 중인 책입니다.")
        return

    users[user].append(title)
    books[title] -= 1

    print(f"[BORROW SUCCESS] {user}님이 '{title}'을(를) 대출했습니다.")

# 책 반납

```

```

def return_book(command):

    if len(command) != 3:
        print("ERROR : 잘못된 명령어 형식입니다.")
        return

    user = command[1]
    title = command[2]

    # 사용자 존재 여부 확인
    if user not in users:
        print("ERROR : 존재하지 않는 사용자입니다.")
        return

    # 실제 대출 여부 확인
    if title not in users[user]:
        print("ERROR : 대출하지 않은 책입니다.")
        return

    users[user].remove(title)
    books[title] += 1

    print(f"[RETURN SUCCESS] {user}님이 '{title}'을(를) 반납했습니다.")

# 책 남은 수량 조회
def check_status(command):

    if len(command) != 2:
        print("ERROR : 잘못된 명령어 형식입니다.")
        return

    title = command[1]

    if title not in books:
        print("ERROR : 존재하지 않는 책입니다.")
        return

    print(f"{title} : {books[title]}권 남음")

# 사용자 대출 목록 조회
def show_user(command):

    if len(command) != 2:
        print("ERROR : 잘못된 명령어 형식입니다.")
        return

    name = command[1]

```

```

if name not in users:
    print("ERROR : 존재하지 않는 사용자입니다.")
    return

if len(users[name]) == 0:
    print("EMPTY")
    return

print(f"{name}님의 대출 목록")

for book in users[name]:
    print(f"- {book}")

# 전체 책 목록 출력
def show_book_list(command):

    if len(command) != 1:
        print("ERROR : 잘못된 명령어 형식입니다.")
        return

    if len(book_order) == 0:
        print("EMPTY")
        return

    print("===== BOOK LIST =====")

    # 등록 순서대로 출력
    for book in book_order:
        print(f"{book} : {books[book]}권")

    print("=====")

# 프로그램 실행
def main():

    print("===== 도서 대출 관리 시스템 =====")
    print("사용 가능한 명령어")
    print("add")
    print("borrow")
    print("return")
    print("status")
    print("user")
    print("list")
    print("Exit")
    print("=====")

    # Exit 입력 전까지 반복
    while True:

```

```

command = input(">> ").split()

if len(command) == 0:
    print("ERROR : 명령어를 입력하세요.")
    continue

if command[0] == "Exit":
    print("프로그램을 종료합니다.")
    break

elif command[0] == "add":
    add_book(command)

elif command[0] == "borrow":
    borrow_book(command)

elif command[0] == "return":
    return_book(command)

elif command[0] == "status":
    check_status(command)

elif command[0] == "user":
    show_user(command)

elif command[0] == "list":
    show_book_list(command)

else:
    print("ERROR : 존재하지 않는 명령어입니다.")

main()

```

5. LLM 활용 및 코드 개선 과정에 대한 고찰

이번 과제를 수행하면서 LLM 이 요구사항을 기반으로 비교적 완성도 높은 코드를 생성할 수 있다는 점을 확인할 수 있었다. 입력한 프롬프트만으로 도서 등록, 대출, 반납, 조회 기능을 대부분 구현하였으며, 함수를 이용한 기능 분리와 자료구조의 선택도 적절하게 이루어져 있었다.

하지만 실제 프로그램을 실행하면서 사용자의 입장에서 프로그램을 사용하는 과정에서는 부족한 부분도 발견할 수 있었다. 특히 성공적으로 명령이

수행되어도 별도의 안내가 없어 프로그램이 정상적으로 실행되었는지 확인하기 어려웠으며, 출력 형식 또한 단순하여 가독성이 다소 떨어졌다.

또한 프로그램의 실행 구조와 주석이 부족하여 다른 사람이 코드를 읽고 이해하기에는 다소 불편한 점이 있었다. 이러한 부분을 직접 수정하면서 프로그램의 사용성과 유지보수성을 높일 수 있었다.

이번 과제를 통해 LLM은 코드를 빠르게 작성하는 데 매우 유용한 도구이지만, 생성된 코드를 그대로 사용하는 것보다는 직접 실행하고 테스트하면서 부족한 부분을 개선하는 과정이 반드시 필요하다는 점을 알게 되었다. 특히 예외 상황을 확인하고 사용자 편의성을 고려하여 출력 형식을 개선하는 과정이 프로그램의 완성도를 높이는 데 중요한 역할을 한다는 것을 경험하였다.

따라서 LLM은 개발을 보조하는 강력한 도구이지만, 최종적인 코드의 품질은 개발자가 직접 검토하고 수정하는 과정에서 결정된다는 점을 이번 과제를 통해 확인할 수 있었다.